

PRINCIPLES OF PROGRAMMING LANGUAGES / LESSON GUIDE

05 Parameter passing

Module 3: subprogram interfaces, aliasing, and ownership**1. Introduction & core concept**

Parameter passing specifies how a function's formal parameters obtain access to the values or objects supplied by its caller. Arguments are the expressions at the call site; parameters are the bindings declared by the function. The central questions are what is copied, what is shared, and which changes can be observed after the call returns.

Separate rebinding from mutation. Rebinding makes a local name denote another value or object. Mutation changes an existing object that may be reachable through several names. Aliasing occurs when multiple access paths reach the same object. Good interfaces make permitted mutation and ownership transfer clear rather than forcing callers to infer them from implementation details.

2. Under the hood: the mechanics

A call evaluates arguments and initializes parameter bindings according to the language's rules. Python parameters are local names bound to supplied objects; object mutation can be visible through aliases [1]. JavaScript copies argument values into parameters; when the value identifies an object, caller and callee can access that same object. Neither model makes reassignment of a parameter automatically reassign the caller's variable.

C++ exposes value and reference parameter types and ownership-bearing objects. Passing an `int` by value creates a separate value; passing `int&` gives an alias. A `unique_ptr` passed by value can transfer ownership when initialized from a moved argument. Calling conventions and registers implement these semantics, but a particular register layout is not the language definition.

Learning outcomes

- Predict mutation versus rebinding effects.
- Recognize aliases across a call boundary.
- Explain C++ value, reference, and ownership-transfer interfaces.

Examples progress from Python 3 to modern JavaScript to C++ 17. Code follows each language's native syntax. Bracketed citations refer to the final page.

3. CODE EXAMPLES / PYTHON

Simple: mutate a list, then rebind the parameter

```
def change(items):  
    items.append(2)  
    items = [99]  
    return items  
  
values = [1]  
replacement = change(values)  
print(values)  
print(replacement)
```

In-depth reasoning

Expected result: [1, 2]
[99]

Before the call, `values` reaches one list containing 1. The parameter `items` is bound to that same object. `append` changes its contents, so the caller later sees [1, 2]. This observation requires a shared mutable object; it does not require direct access to the caller's variable binding.

The assignment `items = [99]` creates another list and rebinds only the local parameter. `values` continues to refer to the original list. Returning `items` gives the caller access to the new list, which it stores in `replacement`. Both objects can now be observed through distinct bindings.

If the function returned nothing explicitly, the mutation would still be visible, but the caller would not receive this replacement list. A shallow copy of the outer list could isolate `append` while still sharing nested objects. Copy depth is a separate design decision from the parameter-passing rule.

Compare the language models

JavaScript object/array arguments show analogous mutation-versus-rebinding behavior. C++ by-value containers normally copy their elements, while a reference parameter aliases the original container. Do not translate Python's behavior into the blanket phrase 'everything is passed by reference'.

Checkpoint: Which statement affects `values`: `append` or parameter assignment? `append` mutates the shared list; parameter assignment does not redirect `values`.

3. CODE EXAMPLES / JAVASCRIPT

Intermediate: shared object properties

```
function revise(record) {  
    record.score += 5;  
    record = { score: 0 };  
    return record;  
}  
  
const student = { score: 10 };  
const replacement = revise(student);  
console.log(student.score);  
console.log(replacement.score);  
console.log(student === replacement);
```

In-depth reasoning

Expected result: 15

0

false

The student binding holds access to an object with score 10. The call initializes record with the same object value. Updating record.score changes that object's property to 15. The subsequent assignment makes record identify a newly created object, so it stops using the original one.

Returning record exposes the second object, which replacement receives. Strict equality on these object operands compares identity, so the final result is false. Their contents need not differ for distinct objects to compare unequal. The const declarations protect the caller's bindings against reassignment, not the objects against property updates.

The design can be made easier to reason about by choosing either a clearly mutating function or a function that constructs and returns a new result. Mixing both, as this teaching example does, is legal but makes effects less obvious. A spread copy of a record also remains shallow when nested objects are present.

Compare the language models

Python can express the same distinction with dictionaries. C++ const references restrict modification through that reference, while ordinary references permit it. JavaScript const is a binding restriction and is not equivalent to a deep const-qualified object graph.

Checkpoint: If record.score were unchanged but record were rebound, would student change? No; only the local parameter's value would change.

3. CODE EXAMPLES / C++

Advanced: value, alias, and ownership

```
#include <iostream>
#include <memory>
#include <utility>

void by_value(int value) { value = 99; }
void by_reference(int& value) { value = 99; }

void consume(std::unique_ptr<int> value) {
    std::cout << *value << '\n';
}

int main() {
    int value = 1;
    by_value(value);
    std::cout << value << '\n';
    by_reference(value);
    std::cout << value << '\n';
    auto owner = std::make_unique<int>(7);
    consume(std::move(owner));
    std::cout << std::boolalpha << (owner == nullptr) << '\n';
}
```

In-depth reasoning

Expected result: 1

99

7

true

`by_value` initializes a separate integer parameter; assigning 99 there leaves the caller's value equal to 1. `by_reference` binds an alias to the caller's integer, so the same assignment changes it to 99. The parameter type makes this distinction part of the function interface.

`make_unique` creates an integer managed by one `unique_ptr`. `std::move` does not itself move data: it enables move construction of the `consume` parameter. `unique_ptr`'s move transfers ownership and leaves `owner` empty. `consume` requires a non-null pointer in this example, prints 7, and releases the managed integer when its owning parameter is destroyed [3].

Copying `unique_ptr` by passing `owner` without a move is not allowed. This restriction prevents two independent unique owners from deleting the same object. The source pointer remains a valid empty object after transfer, so comparing it with `nullptr` is valid; dereferencing it would not be. Other moved-from types may have different guarantees.

Compare the language models

Python and JavaScript usually rely on managed object lifetimes and do not express unique ownership through these parameter types. C++ requires programmers to distinguish borrowing, copying, and transferring ownership, particularly when callbacks or resources outlive a call.

Checkpoint: Why should `consume`'s non-null precondition be documented? `unique_ptr` can legally be empty, but dereferencing an empty one is invalid.

4. CONSOLIDATION / REFERENCES

Reason, compare, and apply

The central lesson

At every call, draw the parameter binding separately from the object it reaches. Then classify each statement as rebinding, mutation, copying, or ownership transfer. The Paradigm Diagnostics lesson compares container mutation with C++ reference assignment; those are related mechanisms, not identical algorithms.

Review questions with answer guidance

1. Can a by-value argument still provide access to a shared object?

Yes. The copied value can be a reference-like or pointer value that reaches the same object.

2. What is a useful alternative to output mutation?

Return a new result explicitly when callers do not need in-place modification.

A short learning activity

Predict each example's outcome before running it. Change one input or one operation, state the rule you expect to apply, and compare the observation with the prediction. Explain the earliest point where behavior changes. A matching output supports that particular execution; it does not prove general program equivalence.

References and further reading

[1] Python FAQ: arguments and output parameters. [Read source](#)

[2] ECMAScript: function definitions. [Read source](#)

[3] C++ working draft: `unique_ptr`. [Read source](#)

References are official language/runtime documentation or the public C++ working draft. Links checked 30 September 2026. Examples and reasoning are original teaching material; implementation details and scheduling outcomes are identified where relevant.